AIAP REPORT

Prepared by Ilamaran Raju PA33 (S10257054C)

TABLE OF CONTENTS

INTRODUCTION
PROBLEM STATEMENT
SYSTEM REQUIREMENTS
SYSTEM ARCHITECTURE
PROTOTYPE
IMPLEMENTION
REFLECTION
REFERANCES

INTRODUCTION

Vacuum cleaners have become indispensable in a fast-paced, clean-conscious society like Singapore. Their daily sweep not only gives floors a spotless finish but cuts down on dust, dander, and other airborne allergens, fostering a healthier indoor environment. However, even the most trusted models run into the same old nuisance: a blocked airway. Most of the time, a stubborn ring of hair or a forgotten crumpled paper clip is to blame, alongside the occasional wayward bulk item that thinks it can hide in the dustbin. Tackling these minor blockages is a quick, satisfying win that costs only a few moments and no parts. But if the vacuum is left to wheeze through the strain, those easy fixes can turn into motor strain and pricey repairs.

PROBLEM STATEMENT

Most vacuum cleaners are built around a universal motor driving a centrifugal fan that pulls air and debris into a collection chamber. If a blockage forms—say, a collapsed hose or a saturated filter—the fan keeps spinning and a small vacuum persists. But because air can’t enter the system, the motor no longer moves particles, and the suction isn’t productive.

Even with the load lightened, there is a more sinister problem, Air flowing through the cleaner also acts as a coolant, carrying away heat generated by motor. Blocked paths mean little or no air past the motor, and heat accumulates unchecked. Each winding then exceeds its rated temperature, softening insulation and shortening the motor’s life—if it survives at all. (When you close off the mouth of a vacuum cleaner, is it using more energy? (2017, June))

So, the core hazard isn’t mechanical wear; it’s the trapped heat. Clog-detection circuitry and continuous airflow monitoring are not optional, they are the safeguards that keep the motor cool, the cleaner running, and the warranty from expiring at the first hidden obstruction.

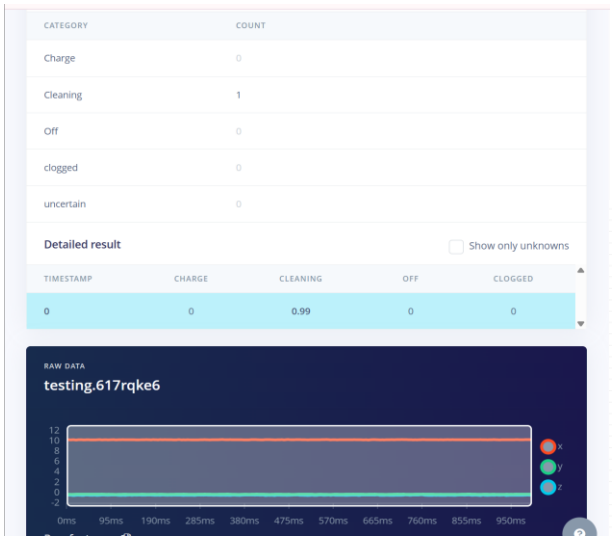
SYSTEM REQUIREMENT

This system uses an MPU6050 sensor to detect motion and vibration inside the vacuum cleaner. The sensor continuously gathers data during operation, allowing a pre-trained AI model to classify the behaviour using supervised learning. Based on it, the AI generates specific labels such as "Charging", "off", or "clogged". (Figure1)

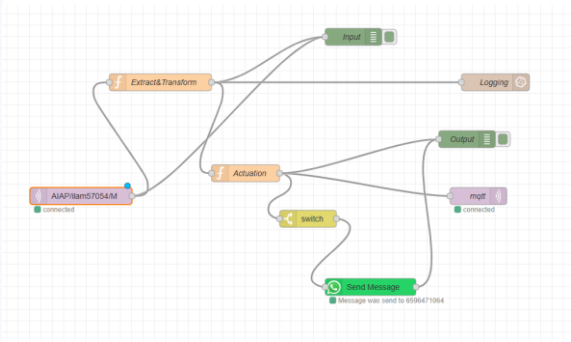
These labels are then sent wirelessly to the cloud, where they are checked to confirm the device's current state.(Figure 2) If a "clogged" condition is detected — usually caused by hair, dust, or thick debris — the system automatically sends a WhatsApp notification to alert the user, helping prevent performance issues before they worsen.(Figure3)

Regardless of the condition, the label is always sent back to the vacuum cleaner, where it is used to update the user locally. A buzzer (speaker) provides audio alerts, LEDs visually indicate status with different colours, and an OLED screen displays the specific label for clarity. This ensures users are always aware of the vacuum's condition without needing to check manually. (Figure 4)

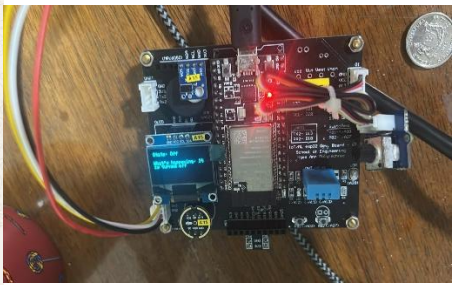
Furthermore, every state detected by the AI is logged and visualized using Grafana, giving users access to detailed historical data. This helps identify recurring problems — such as frequent clogging or unusual vibration patterns — allowing for better preventive maintenance and performance optimization over time.(figure 5)

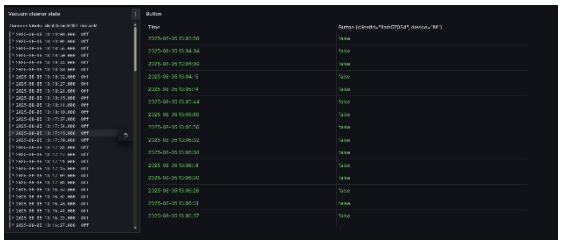


(Figure 1)



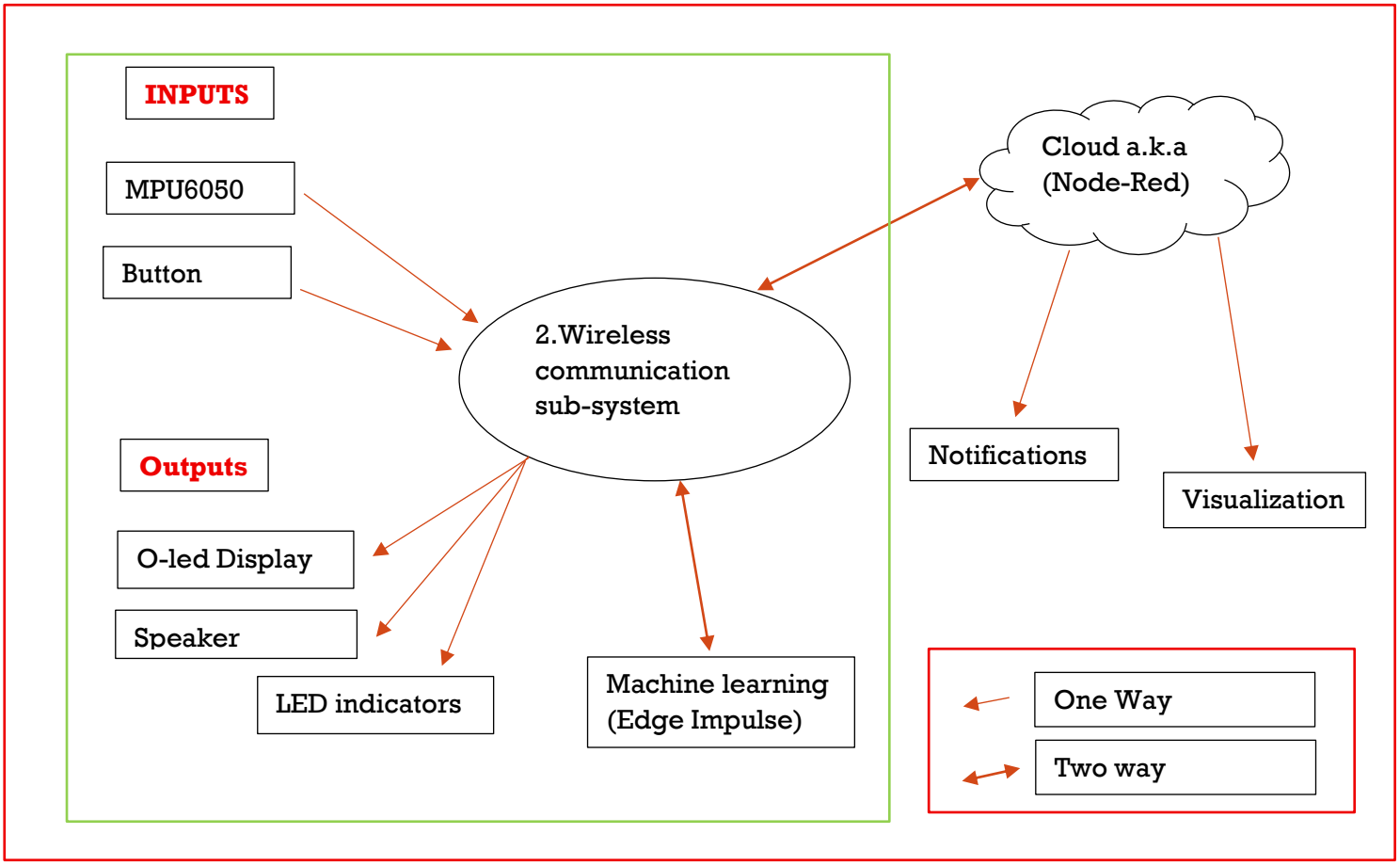
(Figure 2)





(Figure 3) (Figure 4) (Figure 5)

SYSTEM ARCHITECTURE



1. **MPU6050** -The **MPU6050** measures vibrations and movements coming from the vacuum cleaner.
2. **Button-** A button is added to simulate a clogging clearing feature

3. **O-led Display** -The OLED display shows the current state of the vacuum cleaner

4. **Speaker** -The speaker turns on only when the vacuum is blocked and button is pressed (simulating vacuum running in 150 percent to try clear the clogg), signalling the user with a brief tone. This delayed, focused audio cue lets the operator know that prompt action is required.

5. **LED Lights** –The vacuum uses coloured LEDs to show what's happening inside it.

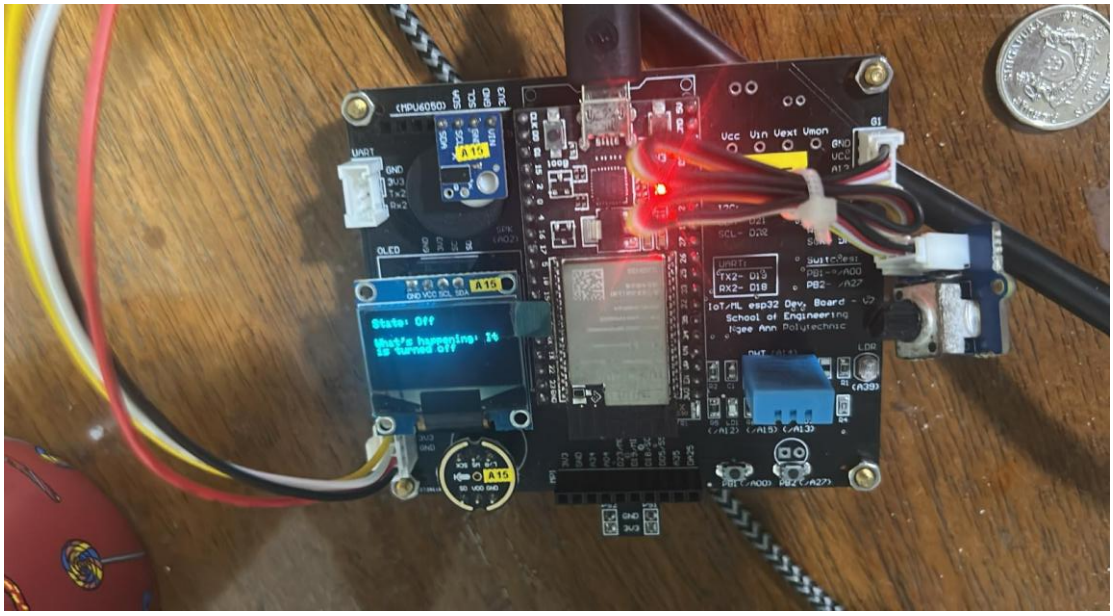
6. **Machine learning (Edge Impulse)** - We leverage Edge Impulse's machine learning framework to analyse motion data from the MPU6050. The model, once trained, is able to classify motion patterns and flag conditions such as "clogged" or "normal" according to the learned thresholds.

7. **Cloud (Node-Red)** – The classified label travels to the cloud through Node-RED, which then checks the vacuum's current state. When the state returns as "clogged," Node-RED immediately fires off a WhatsApp message

8. **Notifications (WhatsApp)** – When the vacuum is detected as clogged, a WhatsApp message is automatically sent to notify the user. This ensures quick action can be taken even if the user is not nearby.

9. **Visualization (Grafana)** - Every detected state gets recorded and shown in Grafana, enabling continuous monitoring and analysis. This setup lets users see trends and evaluate how the vacuum performs as time goes on.

PROTOTYPE



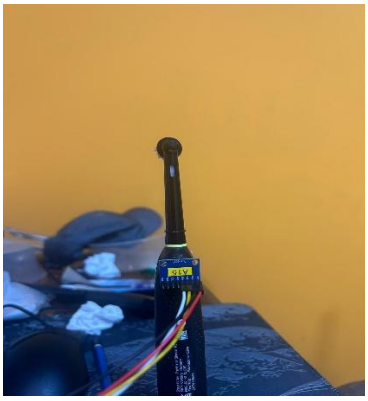
DATA COLLECTION FOR ML

Collection of Data

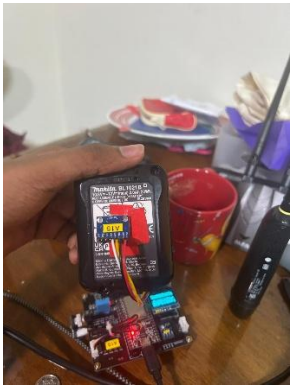
To gather the training data for the machine learning model, I created a variety of vacuum-cleaner scenarios using gentle and playful techniques. For the "normal cleaning" label, I let the vacuum run in its everyday mode and logged vibration readings from the MPU6050 sensor. (Figure 1)To mimic a "clogged" situation without harming the unit, I placed an electric toothbrush against the Mpu6050 and measured the emitted frequency. (Figure 2) When the vacuum was plugged in, I captured the vibration signatures during the battery charge cycle for the "charging" label. (Figure 3) I also logged observations when the vacuum was powered off, ensuring that I recorded its calm, stationary state.

I aimed for a consistent and representative dataset, so I acquired a total of 10 samples per class. Each recording ran for 30 seconds, proportioned collection of data to train the machine learning model effectively.

(Figure 1)



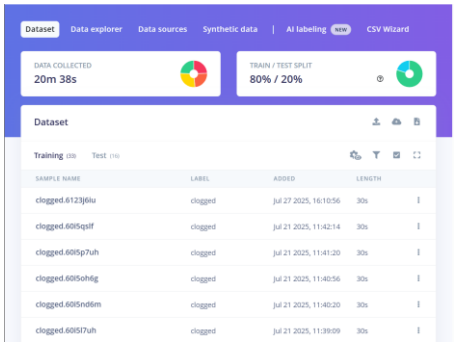
(Figure 2)



(Figure 3)

Splitting the Data

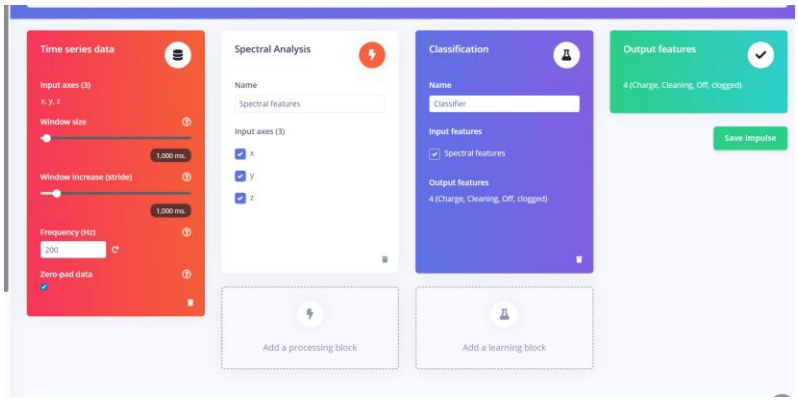
For splitting the collected dataset, I opted for an 80-20 split, where 80% of the data was used for training and 20% for testing. This is a common and effective approach in supervised learning, as it provides the model with enough data to learn patterns while reserving a separate portion to evaluate its performance on unseen data. This helps prevent overfitting . (Figure 1)



(Figure 1)

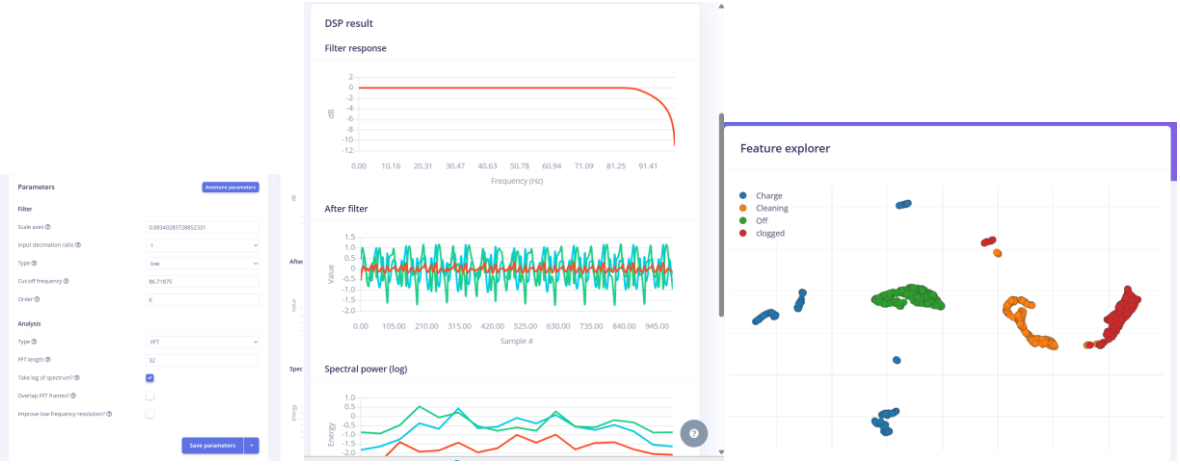
Creating a impulse

In Edge Impulse, I began the impulse design by choosing the X, Y, and Z axes from the MPU6050 sensor for input data. I configured the window length and stride to 1000 ms, allowing the model to check the vibration pattern once every second. With a sampling rate of 200 Hz, the system recorded 200 samples each second, giving a fine-grained look at the vibrations. I turned on zero-padding to keep each data segment the same length, even if some input was a bit shorter. This arrangement produced clean, uniform time-series data that works well for both frequency and classification tasks.



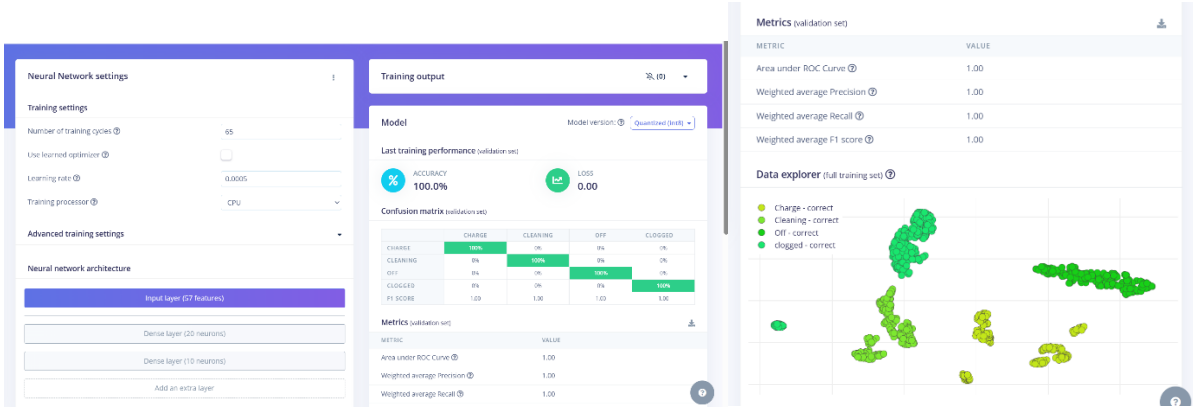
Generating features

To create the features, I applied the Spectral Analysis module using the FFT engine to shift the raw time-series recordings into the frequency domain. Activating the Autotune parameter's function allowed the pipeline to refine the filter design and FFT configuration automatically, optimizing them for the characteristics of my data. The autotune settled on a low-pass filter cutting off at 86.71 Hz and selected an FFT length of 32, capturing only the most informative frequency bands. I chose to log the spectrum to accentuate subtle variations across the different vacuum states, yielding a refined set of frequency features. This configuration ultimately supplied the classification model with clean, distinguishing features for training.



Classifying

For classification, I used a simple neural network architecture with two dense layers (20 and 10 neurons) to keep the model lightweight and efficient for edge deployment. I trained the model over 65 cycles with a low learning rate of 0.0005 to ensure stable convergence. To prevent overfitting, I relied on a balanced dataset (as verified using the Data Explorer) and evaluated the model using the confusion matrix. The matrix showed 100% accuracy on all four classes — *Charge, Cleaning, Off,* and *Clogged* — indicating strong generalization on the validation set. The F1 score and all metrics (ROC AUC, precision, recall) were 1.00, confirming that the model is not just memorizing patterns but learning meaningful features from the spectral data.



Deployment

After this I deployed it and downloaded a zip folder then I included its library from Arduino.

On the device, incoming motion data from the MPU6050 sensor was stored in buffers, matching the same sampling rate and window size used during training. Once a full window of data was collected, it was passed to the Edge Impulse inference engine, which ran the model and returned a classification label (e.g., *Cleaning*, *Off*, *Charge*, or *Clogged*).(fig 1)

```
float buffer[EI_CLASSIFIER_DSP_INPUT_FRAME_SIZE] = {0};
sensors_event_t a, g, temp;

for (int i = 0; i < EI_CLASSIFIER_DSP_INPUT_FRAME_SIZE; i += 3) {
  mpu.getEvent(&a, &g, &temp);
  buffer[i] = a.acceleration.x;
  buffer[i + 1] = a.acceleration.y;
  buffer[i + 2] = a.acceleration.z;
  delay(5);
}

signal_t signal;
numpy::signal_from_buffer(buffer, EI_CLASSIFIER_DSP_INPUT_FRAME_SIZE, &signal);

ei_impulse_result_t result = {0};
EI_IMPULSE_ERROR res = run_classifier(&signal, &result, false);
if (res != EI_IMPULSE_OK) return;

top_label = result.classification[0].label;
float top_score = result.classification[0].value;
for (size_t ix = 1; ix < EI_CLASSIFIER_LABEL_COUNT; ix++) {
  if (result.classification[ix].value > top_score) {
    top_score = result.classification[ix].value;
    top_label = result.classification[ix].label;
  }
}
```

This label was then formatted into a JSON packet and sent wirelessly to the cloud via MQTT, where it was received and processed in Node-RED. Node-RED acted as a central controller — checking the state, triggering actions like WhatsApp notifications, and logging the results to Grafana for long-term monitoring. This deployment flow allowed for real-time, intelligent decision-making using a lightweight, embedded AI system.(fig2)

```
// MQTT Publish
unsigned long currentTime = millis();
if (currentTime - lastPublishTime >= mintxinterval) {
  StaticJsonDocument<200> doc;
  doc["label"] = top_label;
  doc["button"] = buttonPressed;

  char jsonBuffer[200];
  size_t length = serializeJson(doc, jsonBuffer);

  if (length < sizeof(jsonBuffer)) {
    if (client.publish(publish_topic, (const uint8_t*)jsonBuffer, length, false)) {
      Serial.print("Published: ");
      Serial.println(jsonBuffer);
    } else {
      Serial.println("Publish failed");
    }
  }

  lastPublishTime = currentTime;
}
```

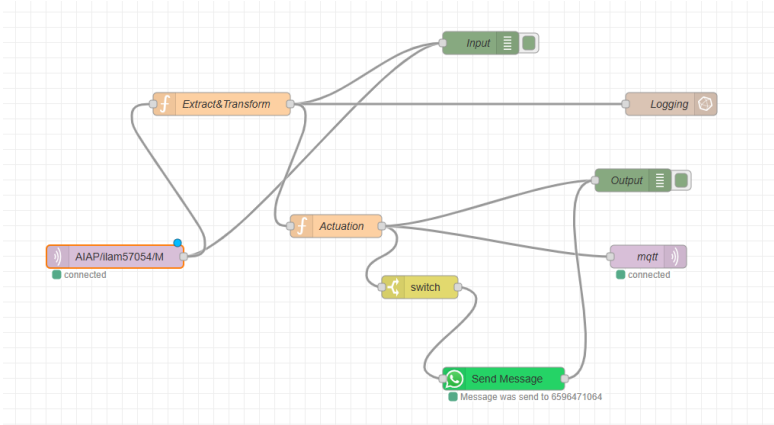
Further more some parts of the codes were written in non blocking manner to ensure the CPU can handle everything at once. For example the the mqtt publish, Inference Classification Loop, check if my button is pressed like the debouncing part

```
unsigned long currentTime = millis();
if (currentTime - lastPublishTime >= mintxinterval) {
```

```
for (int i = 0; i < EI_CLASSIFIER_DSP_INPUT_FRAME_SIZE; i += 3) {
```

```
// Debounce PB1
int reading = digitalRead(PB1);
if (reading != lastButtonState) lastDebounceTime = millis();
if ((millis() - lastDebounceTime) > debounceDelay) buttonPressed = (reading == LOW);
lastButtonState = reading;
```

The data is sent in to node red



Then it is sent to the extract function node where the function begins by pulling out the vacuum state label and the button value from the incoming message payload. Next, it breaks the topic string at each / character to isolate the client ID and device identifiers based on the topic's structure.

With this information in hand, the function assembles a fresh JSON array. The first segment holds the vacuum state and button value, faithfully reflecting the device's latest readings. The second segment packs in the client ID and device name as metadata, clearly marking the source of the data.

The function then hands this reformatted message back to the caller, ready to travel onward in the flow—be it to a node that logs, makes decisions, or presents the information visually.

```
//Assign properties of msg.payload to variables
var state = msg.payload.label
state = state
var button = msg.payload.button
button = button

//Extract info from msg.topic
var result = msg.topic.split("/") //split string to array
//Form New msg.payload as a JSON array
msg.payload = [{ //element 1 > Fields
    "State": state,
    "Button": button,
},
{ //element 2 > Tags
    "clientid": result[1], //2nd element of result
    "device": result[2] //3rd element of result
}];
return msg;
```

Then its split where one part goes to influx db then to grafana to be logged and the other to some internal logic

In the acution function constructs the reply message for the vacuum. It prefixes the original topic with “DL/” to clearly label it as a downlink response. It then pulls the present state (such as “clogged” or “Charge”) and the button value from the incoming payload. Depending on the state and the button press, it produces an appropriate response string.

The routine keeps a copy of the initial state in msg.state in case it is needed later. It then assigns the newly formed response to msg.payload and returns the message, making it ready to be dispatched to the device via MQTT.

```
// Set downlink topic
msg.topic = "DL/" + msg.topic;

// Extract State and Temp
let state = msg.payload[0]?.State;
let button = msg.payload[0]?.Button;

// Build response for MQTT device
let response = "";

if (state === "clogged") {
    if (button == true) {
        response = "150% Power activated";
    } else {
        response = "Clogged";
    }
} else if (state === "Charge") {
    response = "Charging";
} else if (state === "Cleaning") {
    response = "It is ON";
} else if (state === "Off") {
    response = "It is turned off";
} else {
    response = "Unknown state";
}

// Save original state to check later
msg.state = state;

// Set payload for device
msg.payload = response;

return msg;
```

Then the message is sent back to the device where it subscibes to the DL topic

```
// MQTT Setup
void set_topics() {
  sprintf(publish_topic, "%s/%s/%s", appname, clientid, device);
  sprintf(subscribe_topic, "DL/%s/%s/%s", appname, clientid, device);
}
```

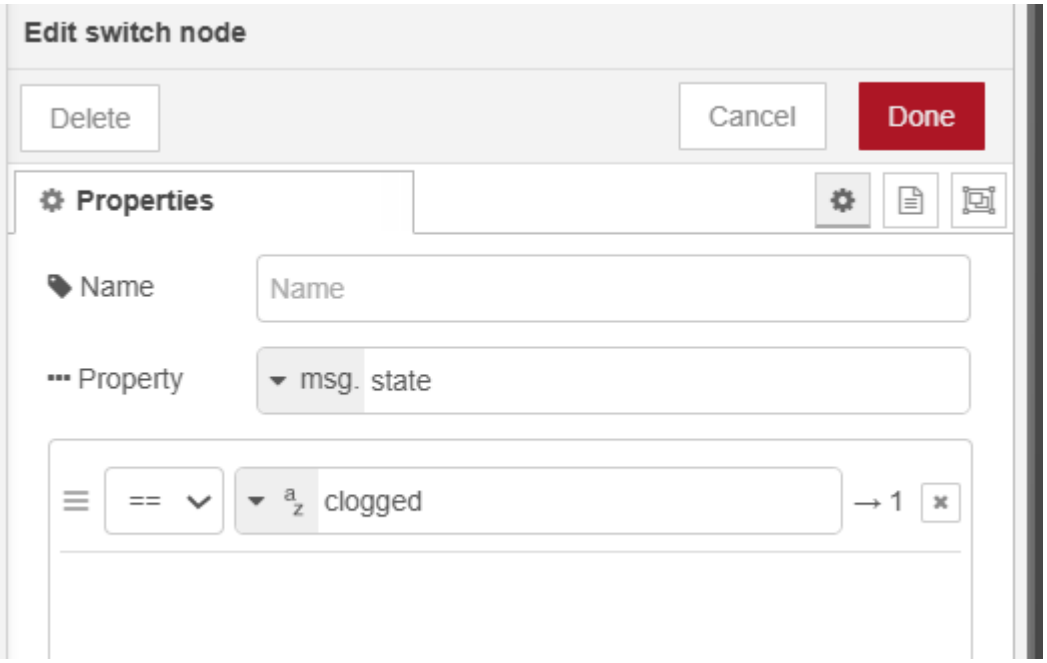
Then it runs the necessary code of the internal logic to light up the led/ buzzer or even the OLED display

```
// MQTT Callback
void callback(char* topic, byte* payload, unsigned int length) {
  received_command = "";
  for (int i = 0; i < length; i++) received_command += (char)payload[i];

  if (String(topic).equalsIgnoreCase(subscribe_topic)) {
    action_state = received_command;

    if (received_command.equalsIgnoreCase("Charging")) {
      digitalWrite(LD1, LOW); digitalWrite(LD2, HIGH); digitalWrite(LD3, HIGH);
      digitalWrite(SPK, LOW);
    }
    else if (received_command.equalsIgnoreCase("Cleaning")) {
      digitalWrite(LD1, HIGH); digitalWrite(LD2, LOW); digitalWrite(LD3, LOW);
      digitalWrite(SPK, LOW);
    }
    else if (received_command.equalsIgnoreCase("Clogged") || received_command.equalsIgnoreCase("150% Power activated")) {
      digitalWrite(LD1, LOW); digitalWrite(LD2, LOW); digitalWrite(LD3, LOW);
      digitalWrite(SPK, HIGH);
    }
    else if (received_command.equalsIgnoreCase("It is turned off")) {
      digitalWrite(LD1, HIGH); digitalWrite(LD2, HIGH); digitalWrite(LD3, HIGH);
      digitalWrite(SPK, HIGH);
    }
  }
}
```

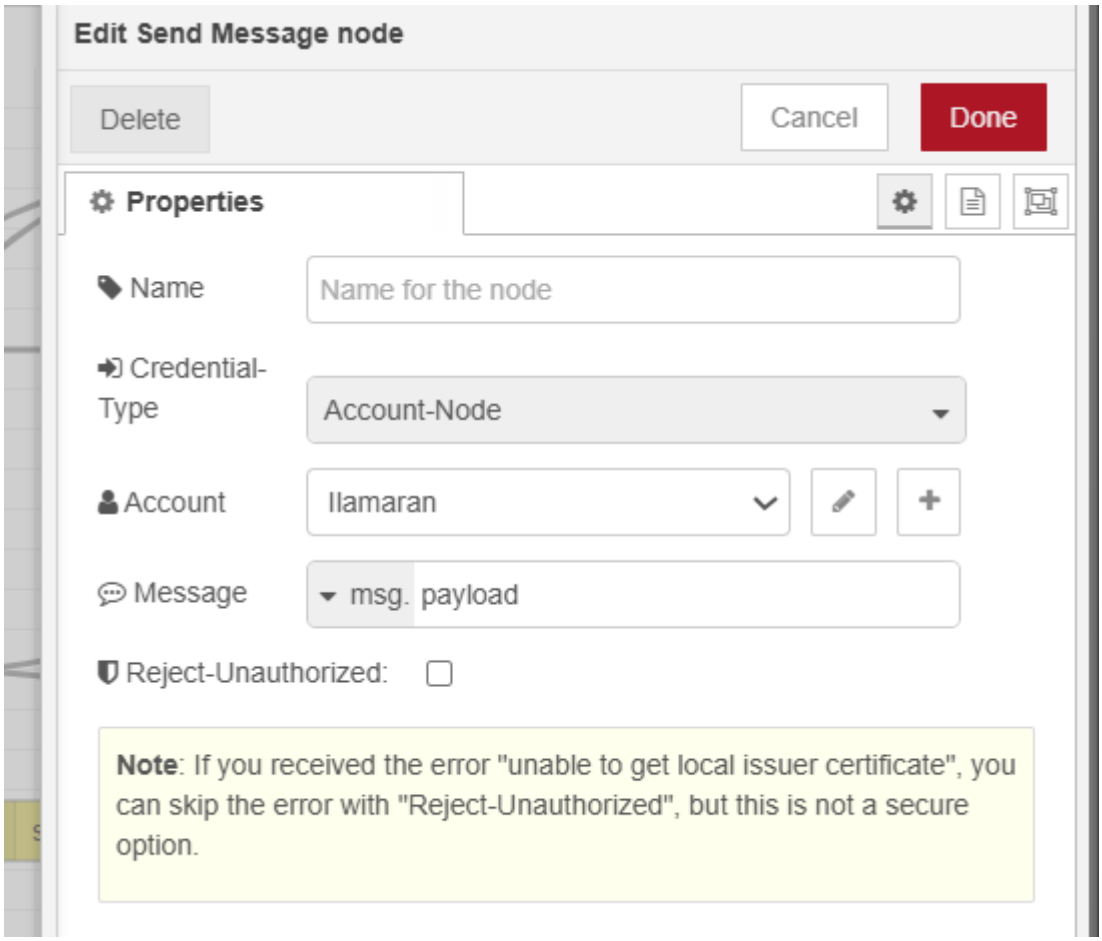
Additional features

I added a feature where if the machine is clogged there will be a notification sent to your phone using a bot

Firstly, I created an account with it installed the necessary applications and made sure I could send a normal message. Then added a switch to filter out the clogged message



Then I sent the msg payload to the WhatsApp which will differ if the button was pressed



So, the result looked like this.

If the button is not pressed and clogged it will remind you its clogged

If its clogged and the button is pressed to simulate a higher suction power is used to try clear the clog and it will message u this

Moreover I added non-blocking code and down link and secure cloud connection in the code

12:40
4G 62
1
Node Red
tap here for contact info
It is turned off 5:56 PM
It is turned off 5:56 PM
It is turned off 5:56 PM
It is turned off 5:57 PM
Resume 11:12 PM
Clogged 11:37 PM
Clogged 11:38 PM
Clogged 11:39 PM
Clogged 11:40 PM
Clogged 11:40 PM
Clogged 11:41 PM
Clogged 11:42 PM
Clogged 11:42 PM
Clogged
150% Power activated
11:42 PM
Clogged 11:42 PM

REFLECTION

Working on the smart vacuum monitoring system was a rewarding ride with a fair number of bumps. The mission was to spot clogged states, using motion data that was classified by AI.

The data collection and feature wrangling part nearly tripped me up. At first, the MPU6050 was a bit stingy; it tossed out numbers, sure, but they didn't differentiate "cleaning" from "off" with any confidence. I quickly understood I did not provide enough useful features for the AI model to clearly distinguish between different vacuum states like "normal" and "clogged" in the code. So, I ramped up buffer limit to capture more data points in my case it was for 1 seconds for a 201herz data.

Putting the system together was another highlight. By choosing Node-RED for cloud communication, connecting WhatsApp alerts, and pushing state logs to Grafana, I felt the project click. Watching the OLED, LEDs, and speaker respond flawlessly to the AI's predictions was rewarding, and every piece felt like it belonged to the same story. It also provided me with nostalgia where I work with in 44iotc it was a confidence boost.

Moreover, Working on this taught me to tame messy sensor readings, the value of careful data prep, and how to marry AI logic with tiny devices. More than the tech, it proved that a common, everyday issue—a vacuum cleaner getting stuck—can spark creative solutions when you mix machine learning with embedded design.

REFERANCES

When you close off the mouth of a vacuum cleaner, is it using more energy? (2017, June). *Stack Exchange*. Retrieved July 22, 2025, from https://physics.stackexchange.com/questions/336694/when-you-close-off-the-mouth-of-a-vacuum-cleaner-is-it-using-more-energy#:~:text=Most%20vacuum%20cleaners%20have%20a,10.9k2%2026%2042